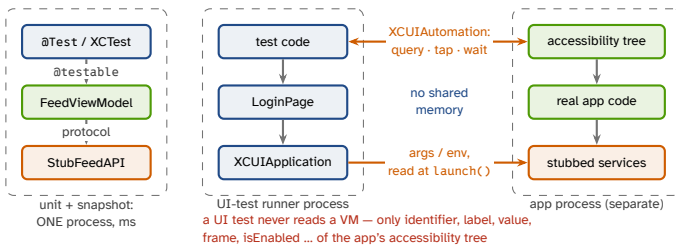
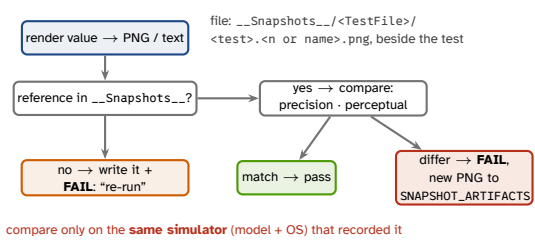


**In one line:** Logic lives in a **ViewModel** and a **router** with injected dependencies, tested in-process in milliseconds; **snapshots** pin what a view looks like (one simulator, one reference file); **UI tests** drive the real app from **another process** through the accessibility tree — few, slow, addressed by **identifier**.

#### Where each test runs



#### assertSnapshot(of:as:)



compare only on the **same simulator** (model + OS) that recorded it

## ViewModel tests

- A SwiftUI View is a value; what it renders is not readable through public API (third-party ViewInspector reflects on the View structs). So: thin View, logic in a VM. Test = **construct with doubles** → **call an input** → **assert state**.
- State as an `Equatable enum` (`idle · loading · loaded · error`) and display strings computed in the VM (`priceText == "$10.00"`); `#expect(a == b)` prints both sides on failure. Inject what varies: API, clock, Date, UUIDs, locale.
- Isolation:** XCTest runs synchronous test methods on the main actor; Swift Testing runs every test on an arbitrary task → a `@MainActor` VM needs a `@MainActor` test or suite.
- Sequences.** `ObservableObject: vm.$state.sink{seen.append($0)}` gets the current value, then each new one — from `willSet`, so `vm.state` read inside the sink is still the *old* value. `@Observable` (Swift 5.9, iOS 17) has no publisher: `withObservationTracking` calls `onChange` once, on the first change — re-register per change. `Observations{ vm.state }` (Swift 6.2, iOS 26) is an `AsyncSequence` of *transactions*: mutations up to the next suspension point coalesce, so values can be skipped — not a recorder of every state.
- To catch .loading, **hold the double**: a stub that suspends until the test releases it; assert mid-flight, then release.
- Big state? `assertSnapshot(of: vm.state, as: .dump)` keeps a reviewed text reference instead of a page of hand-written expectations.

```
@MainActor @Suite struct FeedViewModelTests {
  @Test func offline_showsError() async {
    let vm = FeedViewModel(api: StubFeedAPI(.failure(URLError(.
      notConnectedToInternet))))
    await vm.load()
    #expect(vm.state == .error("Offline"))
  }
  @Test func loading_whileInFlight() async {
    let api = GatedFeedAPI() // suspends until release()
    let vm = FeedViewModel(api: api)
    let load = Task { await vm.load() }
    await api.waitUntilCalled()
    #expect(vm.state == .loading)
    api.release(with: [.sample]); await load.value
    #expect(vm.state == .loaded([.sample]))
  }
}
```

## Coordinator / router tests

- Test the **decision** (which route for which event), never a real push: inject a Router protocol and let a spy record calls.
- SwiftUI: an `@Observable` router owns `var path: [Route]` bound to `NavigationStack(path:)` → assert the array. `NavigationPath` is type-erased — only count, `isEmpty`, `append`, `removeLast`, `codable`; elements can't be read back (it is `Equatable`: compare with a path you build).
- Deep links are pure functions `URL → [Route]`: table-test them with `@Test(arguments:)`, one case per URL.

```
final class SpyRouter: Router {
  private(set) var shown: [Route] = []
  func show(_ r: Route) { shown.append(r) }
  @Test func selectingItem_routesToDetail() {
    let router = SpyRouter()
    FeedCoordinator(router: router).didSelectItem(id: 7)
    #expect(router.shown == [.detail(id: 7)]) // the DECISION
  }
}
```

## Traps

- ! Querying by label: `app.buttons["Log in"]` matches the label, passes in English, fails in a German run. `accessibilityIdentifier` is "for testing", never shown or localized.
- ! `sleep(3)`: flaky when short, slow when long — wait for the element or property.
- ! A gated stub never released hangs the run — give the suite `.timeLimit(.minutes(1))` (minutes are the smallest unit).
- ! UI tests on the real backend: nondeterministic, slow, may mutate data — stub by launch env.
- ! CI on default `.missing`: a lost reference is re-recorded, the run fails, the retry passes → `.never`. Never commit `.all`. "Just re-record" approves a regression unseen.
- ! Reference from another simulator or OS → fails on CI: pin model, OS, locale, appearance, text size; fix dates and IDs.
- ! An interruption monitor for an *expected* alert never fires — tap the alert directly.

## XCUITest toolkit (XCUIAutomation)

| need        | API · fact                                                                                                                                                                                                                                    |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| clean start | <code>launch()</code> : synchronous; <b>terminates</b> a running instance; a launch failure halts the test                                                                                                                                    |
| resume      | <code>activate()</code> : launches if needed, never terminates; reuses the last <code>launch()</code> 's args/env                                                                                                                             |
| configure   | <code>launchArguments / launchEnvironment</code> — changed after launch = next launch only; <code>open(url)</code> launches by URL                                                                                                            |
| find        | <code>app.buttons["id"]</code> matches identifier <b>or</b> title, label, value, placeholder; <code>.element</code> fails unless exactly 1 match; <code>.firstMatch</code> stops at the first (faster)                                        |
| wait        | <code>waitForExistence(timeout:)</code> ; <code>Xcode 16</code> : <code>waitForNonExistence</code> , <code>wait(for: \.value, toEqual:, timeout:)</code> → <code>Bool · exists</code> ≠ <code>isHittable</code> (a hit point can be computed) |
| alerts      | <code>expected</code> → tap it; <code>addUIInterruptionMonitor</code> runs only when unrelated UI blocks an interaction; LIFO; <code>true</code> = cleared                                                                                    |
| state       | <code>resetAuthorizationStatus(for:)</code> (iOS 13.4) · <code>XCUIDevice.shared</code> · <code>.orientation</code> · <code>.appearance</code> · <code>.location</code>                                                                       |
| atly        | <code>try app.performAccessibilityAudit()</code> (iOS 17): fails the test itself; <code>handler true = ignore</code> ; on-screen only · <code>Xcode 27</code> : <code>voiceOverService.moveForward()</code> → speech                          |
| matrix      | <code>runForEachTargetApplicationUIConfiguration = true</code> : <code>appearance × orientation × localization</code> (Xcode 27 launch-test template)                                                                                         |
| perf        | <code>XCTApplicationLaunchMetric · XCHitMetric</code> (iOS 26)                                                                                                                                                                                |
| evidence    | <code>XCTAttachment(screenshot:)</code> default · <code>.deleteOnSuccess</code> ; test plan keeps video + screenshots of <i>failing</i> runs unless "On, and keep all"                                                                        |

**Xcode 26 recording:** the record button at the editor edge writes Swift, each line with alternative queries — prefer identifier over index. Report (Xcode 15+): video replay, Automation Explorer at the failure. **Xcode 27:** app-crash severity off / warning / **failure** / fatal per test plan. Swift Testing has **no** UI automation — UI tests stay XCTest.

```
override func setUpWithError() throws { continueAfterFailure = false }
@MainActor func test_login_showsWelcome() throws {
  let app = XCUIApplication() // the Target Application
  app.launchArguments += ["-AppleLanguages", "(de)"]
  app.launchEnvironment["STUB_SCENARIO"] = "loggedOut"
  app.launch() // args/env read HERE
  LoginPage(app: app).login(email: "a@b.co", password: "pw")
  XCTAssertTrue(app.staticTexts["welcome.title"].waitForExistence(timeout: 5))
  try app.performAccessibilityAudit()
} // Page Object: one type per screen owns its identifiers + actions
```

## Snapshot tests (swift-snapshot-testing)

- `assertSnapshot(of:as:named:record:timeout:)` (timeout 5s). Strategies: `.image` (UIView, VC, SwiftUI), `.recursiveDescription`, `.dump`, `.json`, `.plist`, `.raw` (URLRequest); a text reference shows its diff readably in code review.
- SwiftUI layout: `.sizeThatFits` (default) · `.fixed(width:height:)` · `.device(config: .iPhone13)`; a config is size + safe area + traits only — newest iPhone preset `iPhone13ProMax`. traits: for dark mode / text size; `drawHierarchyInKeyWindow`: true to render UIAppearance and visual effects.
- `precision` = share of pixels that must match; `perceptualPrecision` = how close each pixel must be (CIE LabDeltaE); `0.98–0.99 ≈` the human eye. Both default 1.
- `assertInlineSnapshot` writes a text snapshot into the test source, as a trailing closure.

| record                | behaviour (since 1.17, Jul 2024)                                  |
|-----------------------|-------------------------------------------------------------------|
| <code>.missing</code> | default: record only absent references (and fail)                 |
| <code>.all</code>     | rewrite every reference — every assertion fails                   |
| <code>.failed</code>  | re-record only failing ones (keeps in-threshold passes untouched) |
| <code>.never</code>   | a missing reference is a failure — <b>for CI</b>                  |

Set per call (`record:`), per scope (`withSnapshotTesting(record:)`), per suite (`@Suite(.snapshots(record:))`) or per run (`SNAPSHOT_TESTING_RECORD=never`).

## Which test sees what

| test          | sees                      | breaks on          | misses           |
|---------------|---------------------------|--------------------|------------------|
| VM unit       | state, strings            | logic change       | layout, wiring   |
| router        | routes                    | flow change        | real transitions |
| snapshot      | pixels / text dump        | any pixel, new OS  | behaviour, taps  |
| UI (XCUITest) | atly tree of the real app | copy, timing, data | internals; slow  |